



BITS Pilani

Pilani Campus

Deep Neural Network

Dr S Suresh Kumar

Deep Neural Network



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



- Computer vision tasks
- Image classification challenges
- Convolution Neural Network
- Why CNN for Image data

Computer vision tasks



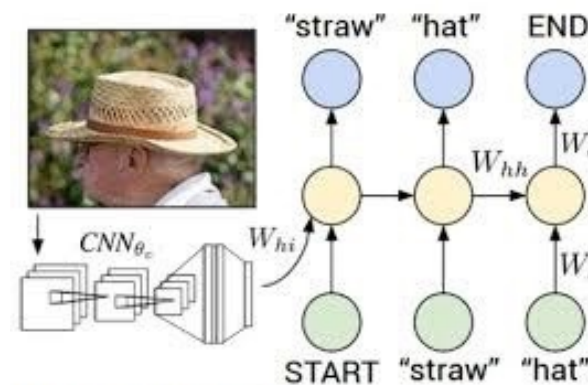
Classification



"SIFT" & Object Recognition, David Lowe, 1999



Image captioning



Andrei Karpathy, Fei-Fei Li

Grayscale Image



Pixel intensities in the greyscale matrix at i^{th} row and j^{th} column: $I(i,j) = [0, 255]$



0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

<https://mozanunal.com/2019/11/img2sh/>

Color Image



- Each pixel now is a vector of 3 values (channels) for Red, Green and Blue (RGB). Each value in this vector varies from 0 to 255
- Total colours that can be produced = $256 \times 256 \times 256 = 1,67,77,216$
- The i th row and j th column: $I(i, j) = [R(i, j), G(i, j), B(i, j)]$
- Image processing like filtering is applied on these images to perform various tasks like sharpening, blurring and edge detection.

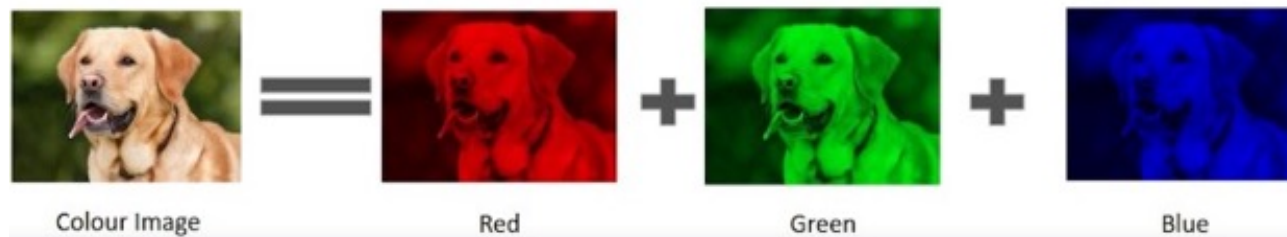


Image classification : A core task in computer vision



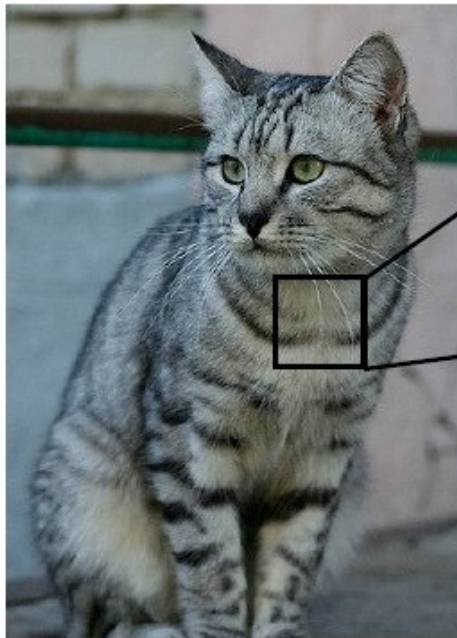
This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

The Problem: Semantic Gap



This image by Nikita is
licensed under [CC-BY 2.0](#)

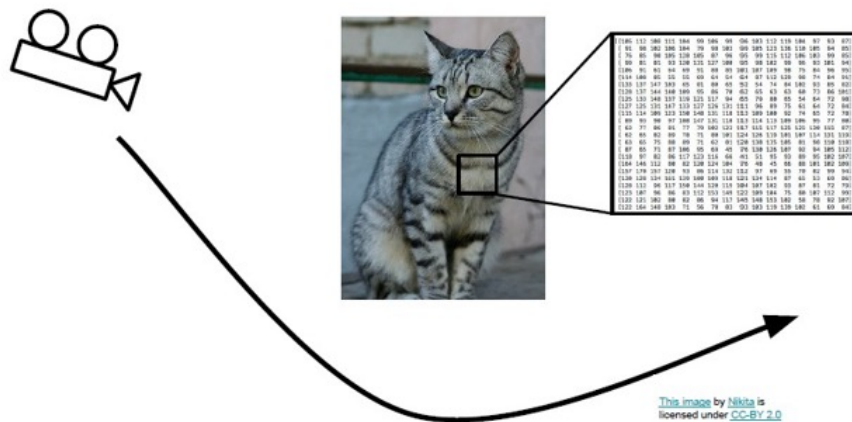
[	185	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
[	91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
[	76	85	90	105	128	105	87	96	95	99	115	112	106	103	99	85]
[	99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
[	106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
[	114	108	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
[	133	137	147	103	65	81	88	65	52	54	74	84	102	93	85	82]
[	128	137	144	140	109	95	86	70	62	65	63	63	68	73	86	101]
[	125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
[	127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
[	115	114	109	123	150	148	131	118	113	109	100	92	74	65	72	78]
[	89	93	90	97	108	147	131	118	113	114	113	109	106	95	77	80]
[	63	77	86	81	77	79	102	123	117	115	117	125	125	138	115	87]
[	62	65	82	89	78	71	88	101	124	126	119	101	107	114	131	119]
[	63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
[	87	65	71	87	106	95	69	45	76	130	126	107	92	94	105	112]
[	118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
[	164	146	112	80	82	128	124	184	76	48	45	66	80	101	102	109]
[	157	170	157	120	93	86	114	132	112	97	69	55	70	82	99	94]
[	130	128	134	161	139	108	109	118	121	134	114	87	65	53	69	86]
[	128	112	96	117	150	144	128	115	184	107	102	93	87	81	72	79]
[	123	107	96	86	83	112	153	149	122	109	104	75	88	107	112	99]
[	122	121	102	80	82	86	94	117	145	148	153	102	58	70	92	107]
[	122	164	148	103	71	56	78	83	83	103	119	139	102	61	69	84]]

What the computer sees

An image is just a big grid of
numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Challenges



All pixels change when the camera moves!

Viewpoint variation



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



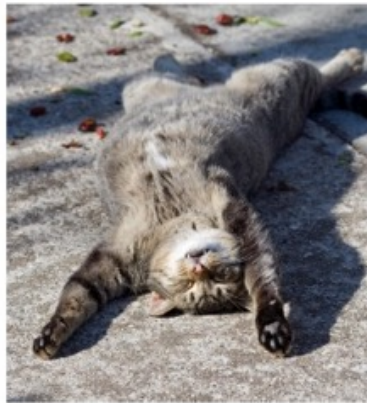
This image is CC0 1.0 public domain

Illumination

Challenges



This image by Umberto Salvagnin is licensed under [CC-BY 2.0](#)



This image by Umberto Salvagnin is licensed under [CC-BY 2.0](#)



This image by rare bear is licensed under [CC-BY 2.0](#)



This image by Tom Thai is licensed under [CC-BY 2.0](#)

Deformation



This image is [CC0 1.0](#) public domain



This image is [CC0 1.0](#) public domain



This image by jonsson is licensed under [CC-BY 2.0](#)

Occlusion

Challenges



Background clutter

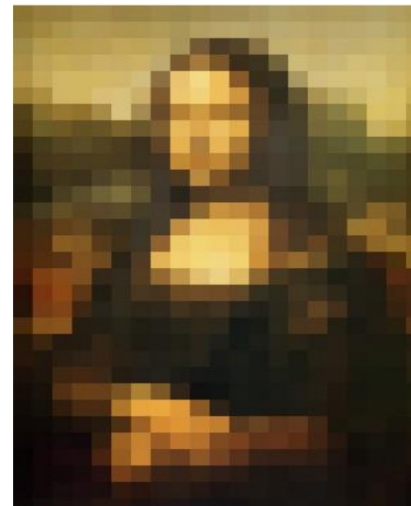


This image is CC0.1.0 public domain

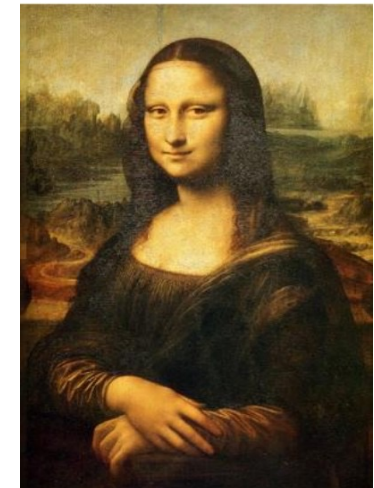


This image is CC0.1.0 public domain

Pixelated images



Larger images



- A 50x50 Image requires 2500 input dimensions.
- A HD image 1920x1080 is about 2 million inputs. (2 Mega Pixels)

Challenges



Inter-class vs intra-class variation



(a) Acura RL Sedan 2012



(b) Acura RL Sedan 2012



(c) Daewoo Nubira W 2002

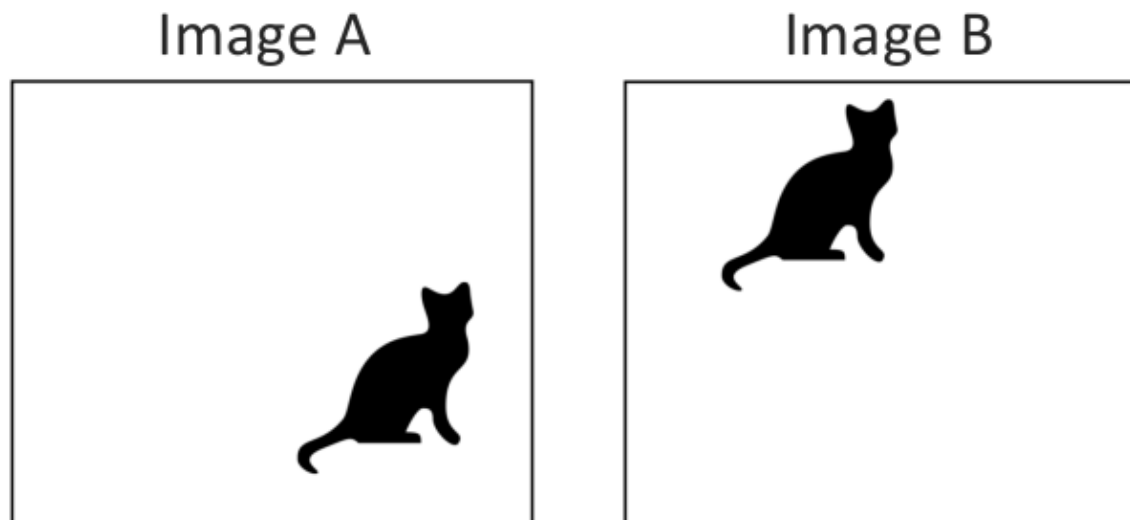


(d) Daewoo Nubira W 2002

Challenges - Translation Invariance



Object can appear anywhere in the image (Max pooling ensures **Translation Invariance**)



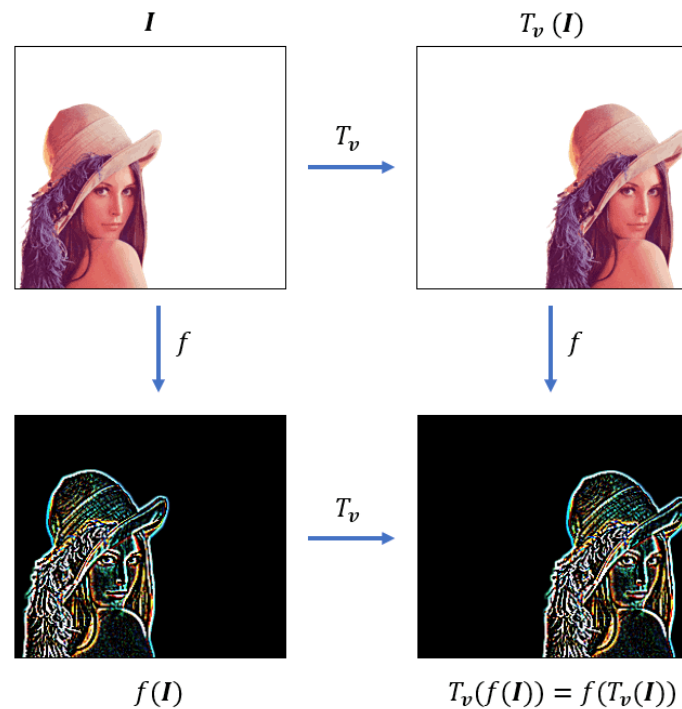
Model predicts “cat” for both images!

Source: <https://penkovsky.com/neural-networks/day5/>

Image Equivariance - Challenges



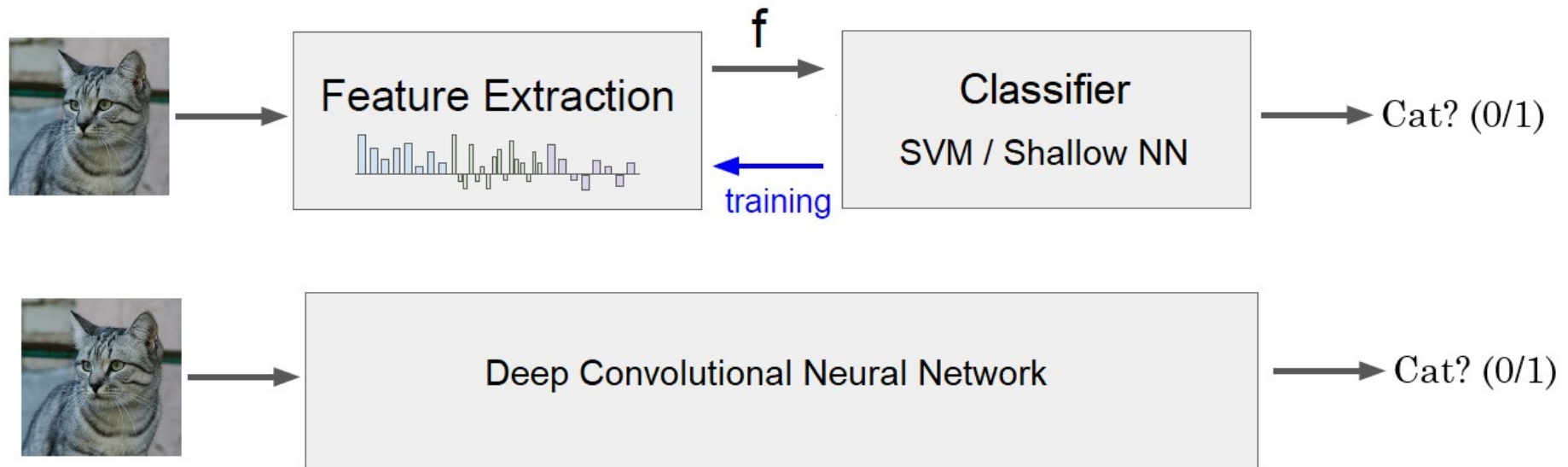
Changes to the input predictably (equivalently) should not affect the output.



$$f(g(x)) = g(f(x))$$

convolution operation is translation equivariant

Image Classification - Solution

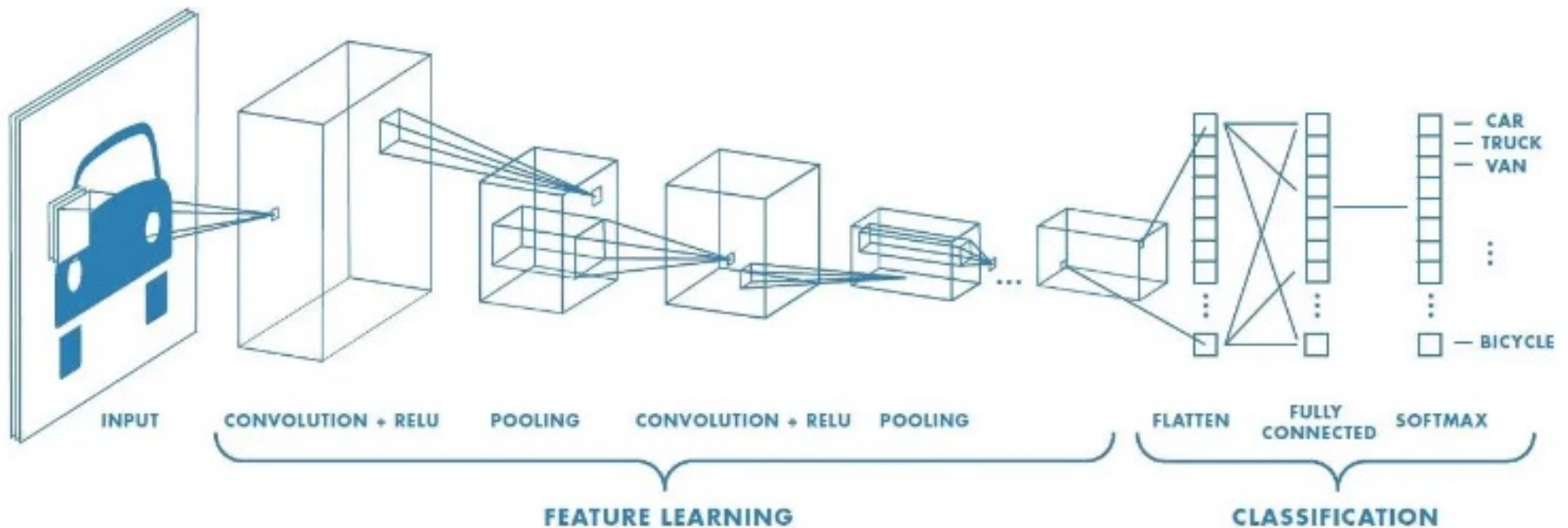


Convolution Neural Network - CNN



- Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers
- Network employs a mathematical operation called **convolution**
- Hierarchical architecture can detect the complex patterns in any area of the visual field
- Studies of the visual cortex inspired the **Neocognitron** in 1980s which gradually evolved into what we now call Convolutional Neural Networks (CNN)
- In 1998, Yann LeCun introduced the **LeNet-5** architecture that introduced new **Convolutional** and **Pooling** layers in the neural networks, which led to CNN

Convolution Neural Network - CNN



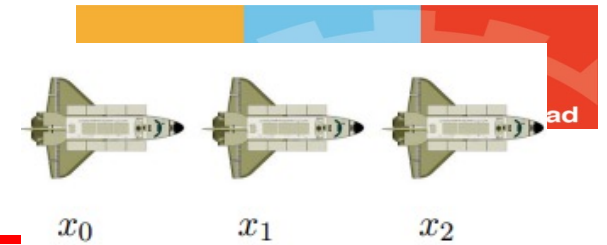
Convolution



- In mathematics, convolution is a mathematical operation on two functions that produces a third function that expresses how the shape of one is modified by the other. [Wikipedia]
- The term convolution refers to both the result function and to the process of computing it. [Wikipedia]
- Special kind of neural network for processing data that has a known, grid-like topology
 - Eg: Time-series data – 1D grid taking samples at regular time intervals.
 - Eg: Image data – 2D grid of pixels

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

1-D Convolution



- Suppose we track position of an object using a sensor or GPS device at discrete time intervals and Assume that the sensor is noisy.
- To obtain a less noisy estimate, we would like to take a weighted average that gives more weight to recent measurements

$x(t)$ is the position of the spaceship at time t - **Input**

$w(a)$ = weighting function, where a is the age of a measurement - **Kernel or filter**

$s(t)$ = a smoothed estimate of the position s of the spaceship - **Feature map**

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t - a)$$

- We would only sum over a small window or a weight array (filter) (w).
- Slide the filter over the input and convolve. The input and the kernel are one dimensional.

1-D Convolution



$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

$$s_6 = x(0).w(6) + x(1).w(5) + x(2).w(4) + x(3).w(3) + x(4).w(2) + x(5).w(1) + x(6).w(0)$$

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80						
---	--	--	--	--	--	------	--	--	--	--	--	--

	w_{-6}	w_{-5}	w_{-4}	w_{-3}	w_{-2}	w_{-1}	w_0
W	0.01	0.01	0.02	0.02	0.04	0.4	0.5

X	1.00	1.10	1.20	1.40	1.70	1.80	1.90	2.10	2.20	2.40	2.50	2.70
---	------	------	------	------	------	------	------	------	------	------	------	------

S						1.80	1.96					
---	--	--	--	--	--	------	------	--	--	--	--	--

2D Discrete Convolution Operation



- Let m and n are kernel height and width respectively

$$S_{ij} = (I * K)_{ij} = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I_{i+a, j+b} K_{a,b}$$

- For 1-index formula will change slightly

$$S_{ij} = \sum_{a=1}^m \sum_{b=1}^n I_{i+a-1, j+b-1} K_{a,b}$$

1	0	1
0	1	0
1	0	1

Filter / Kernel

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature

- Start at top left corner Perform element wise product and add all the values**
- Convolution Operation on an image is performed using a **filter** or **kernel**.

2D Convolution Operation



- Normally both the dimensions of the kernel are same and in odd count like 3x3 or 5x5 so that it can be placed over an image pixel symmetrically
- After the convolution operation, the output is called the **feature** or **activation map**
- Observe that the image size was 5x5, but the output is of size 3x3. It is reduced.
- We will assume that the kernel is square matrix
- **Kernel values are the network parameters to be learned**



Convolution- Complete example



0	1	2
2	2	0
0	1	2

3 ₀	3 ₁	2 ₂	1	0
0 ₂	0 ₂	1 ₀	3	1
3 ₀	1 ₁	2 ₂	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3 ₀	2 ₁	1 ₂	0
0	0 ₂	1 ₂	3 ₀	1
3	1 ₀	2 ₁	2 ₂	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2 ₀	1 ₁	0 ₂
0	0	1 ₂	3 ₂	1 ₀
3	1	2 ₀	2 ₁	3 ₂
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0 ₀	0 ₁	1 ₂	3	1
3 ₂	1 ₂	2 ₀	2	3
2 ₀	0 ₁	0 ₂	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0 ₀	1 ₁	3 ₂	1
3	1 ₂	2 ₂	2 ₀	3
2	0 ₀	0 ₁	2 ₂	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1 ₀	3 ₁	1 ₂
3	1	2 ₀	2 ₁	3 ₀
2	0	0 ₀	2 ₁	2 ₂
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3 ₀	1 ₁	2 ₂	2	3
2 ₂	0 ₂	0 ₀	2	2
2 ₀	0 ₁	0 ₂	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3	1 ₀	2 ₁	2 ₂	3
2	0 ₂	0 ₂	2 ₀	2
2	0 ₀	0 ₁	0 ₂	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

3	3	2	1	0
0	0	1	3	1
3	1	2 ₀	2 ₁	3 ₂
2	0	0 ₂	2 ₂	2 ₀
2	0	0 ₀	0 ₁	1 ₂

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

- Start at top left corner
- Perform element wise product and add all the values.
- Slide by one pixel towards right. Perform element wise product and add all the values
- Slide by one pixel towards right. Perform element wise product and add all the values
- Jump to next row, left most pixel. The filter has to completely superimpose the pixels of input.

CNN- spatial invariance



- **The same patterns appear in different region**
- CNN perform the same exact computation on different parts of the image, i.e. it computes the same features of an image, across all spatial areas

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

After sliding through the entire image.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

CNN – multiple filters for different feature detection



stride $s = 1$ 6 x 6 image

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

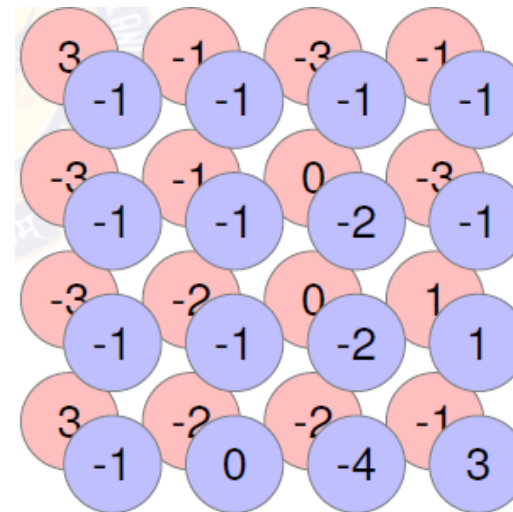
Filter 1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 2

-1	1	-1
-1	1	-1
-1	1	-1

Do the same process for every filter



Property 1

Each filter detects a small pattern (3 x 3)

Edge detection using convolution operation



Vertical edge transitioning from light to dark

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

*

--	--

--	--	--

--	--	--

Sobel filter

1	0	-1
2	0	-2
1	0	-1

Edge detection using convolution operation



Vertical edge transitioning from dark to light

0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

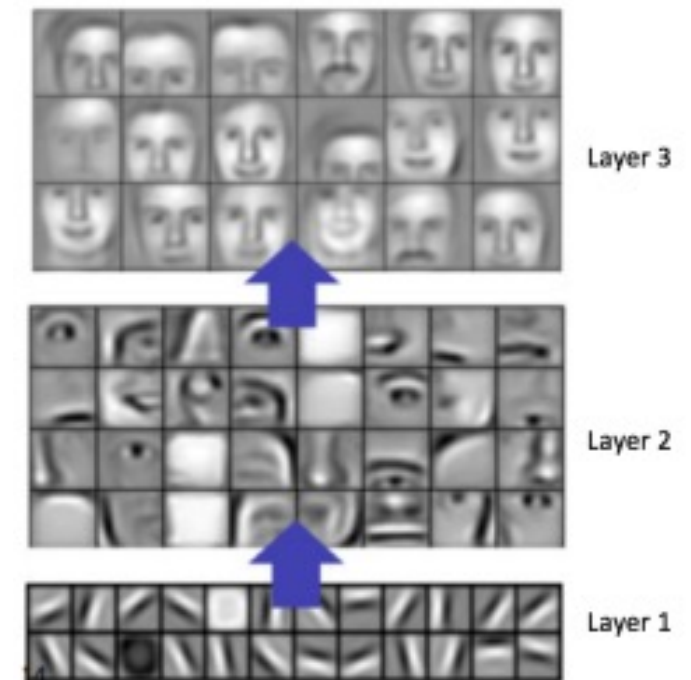
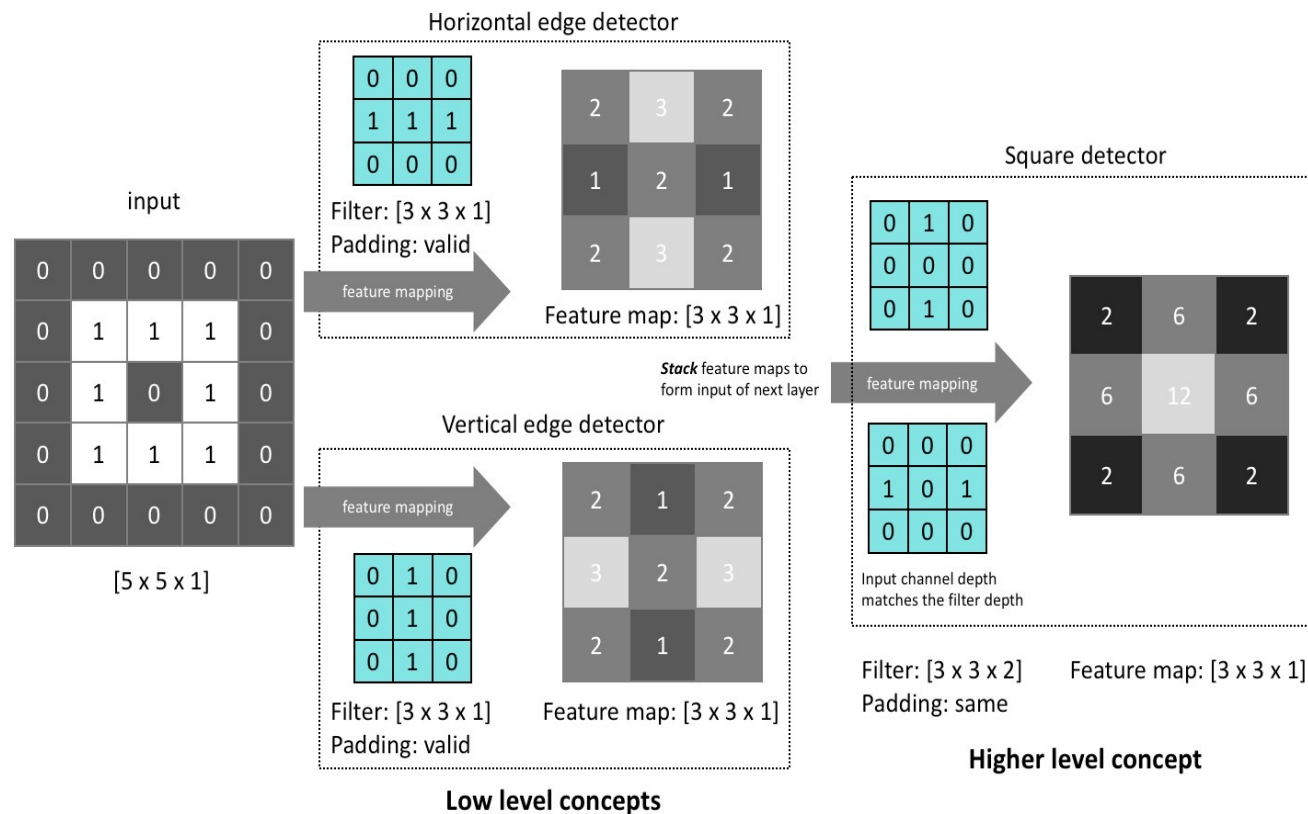
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



Filter values as parameters to be learnt

w1, w2, w3
w4, w5, w6
w7, w8, w9

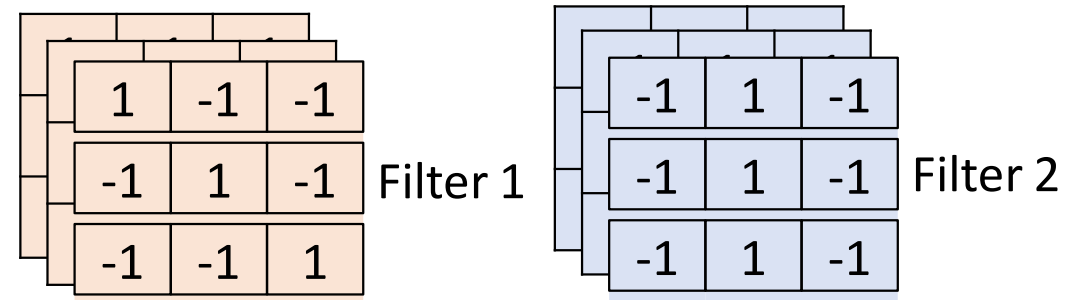
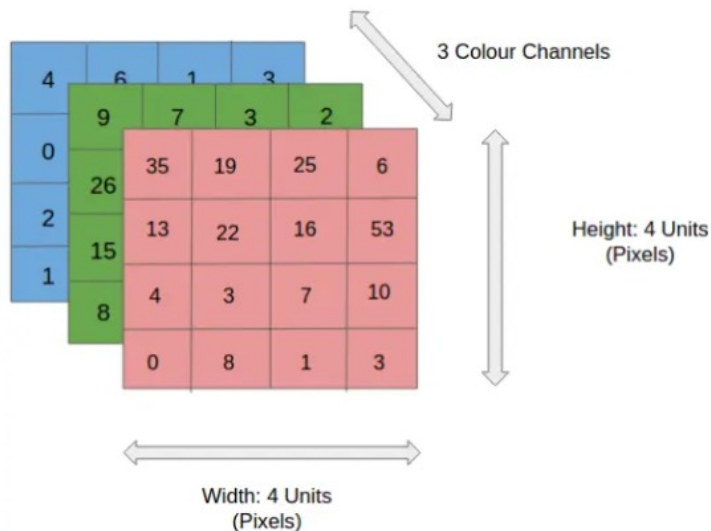
Edge detection using convolution operation



CNN – Colorful image

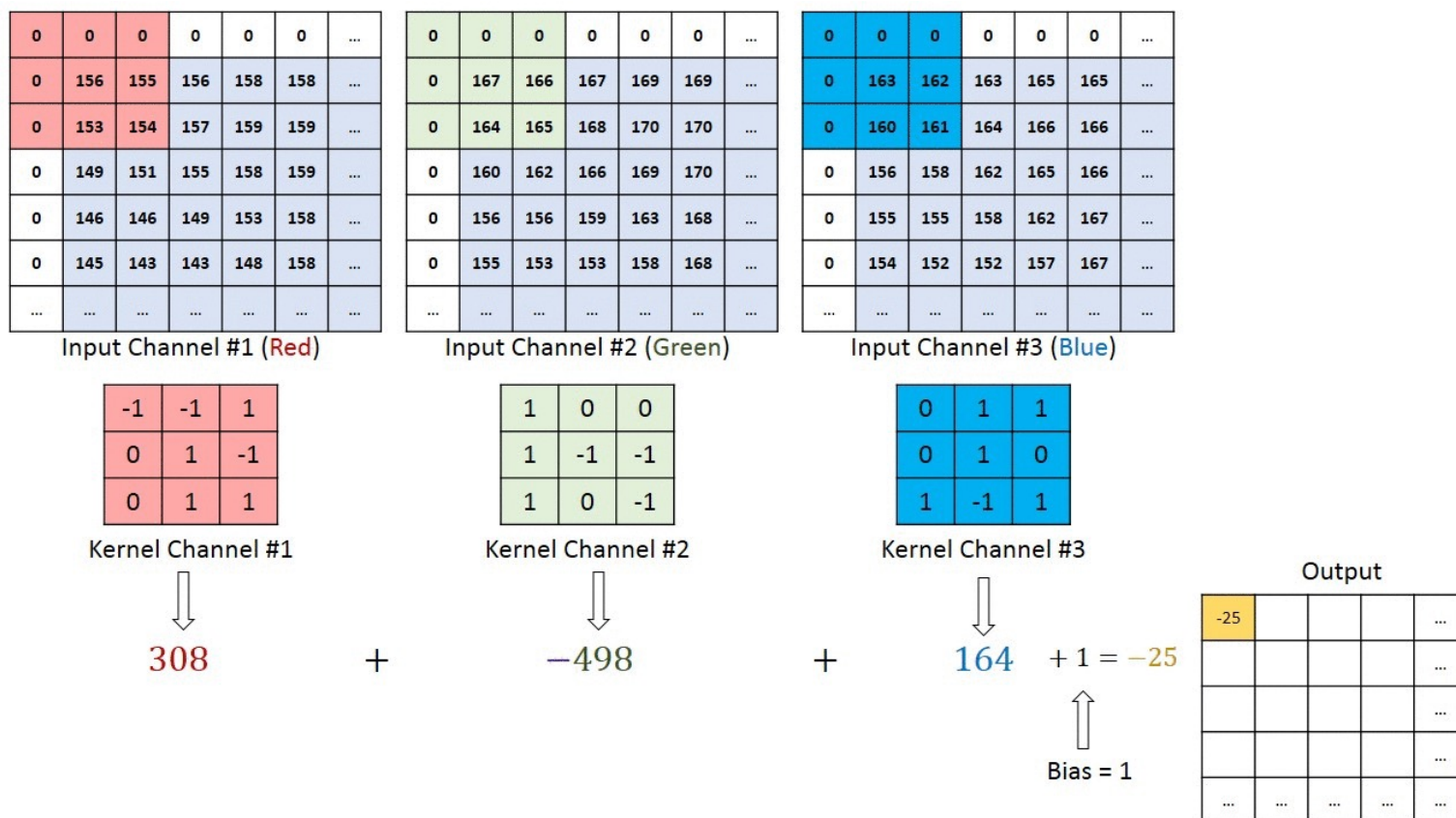


- Convolution operation on colored images (RGB Images) called **convolution over volume**.
- The count of channels in a convolution filter must be same as the count of channels in the input on which convolution is being applied.



Multiple filters can be used over an image!

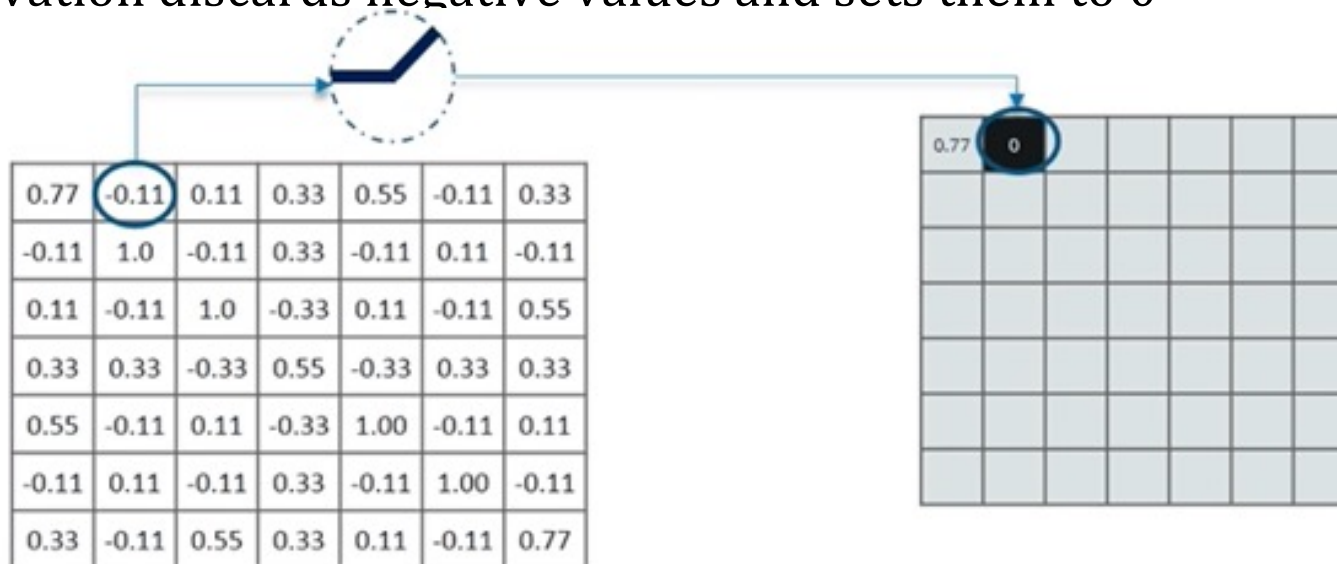
CNN – Colourful image



- Filter cube is sided over the image from the top left corner. The 27 values (3x3x3) of the filter are correspondingly multiplied with the corresponding 27 image channel pixel values to produce a single value
- Repeating the procedure for the entire image yields a 2D output

Relu Activation

- After filtering, the feature maps pass through the activation function
- Activation function is used for scoring pixel values according to some measure of importance
- ReLU activation discards negative values and sets them to 0



Relu in images



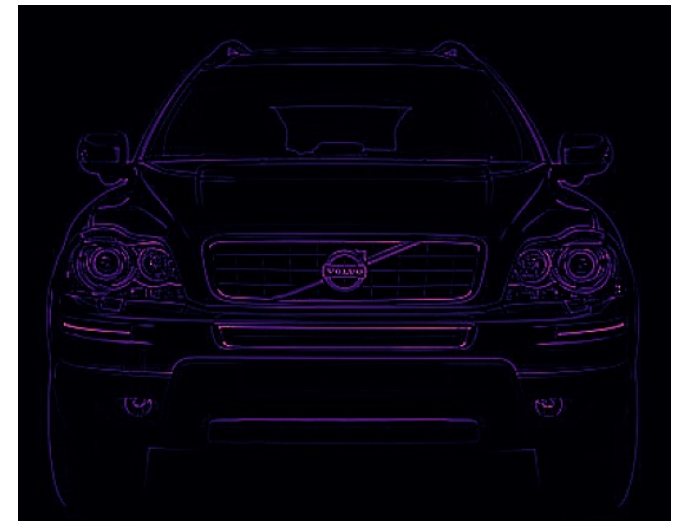
Input image



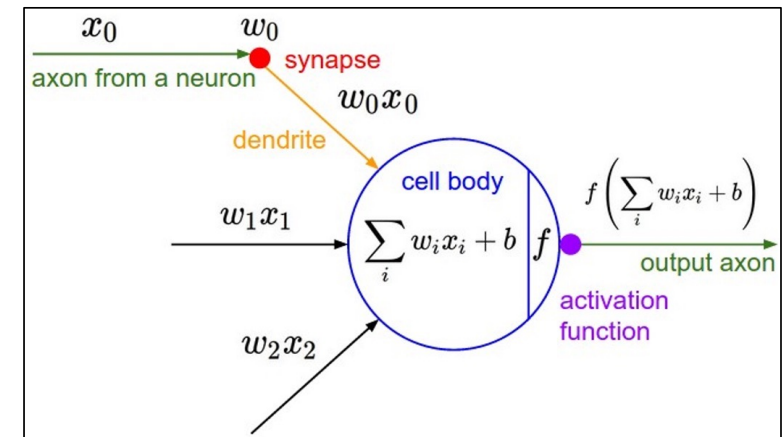
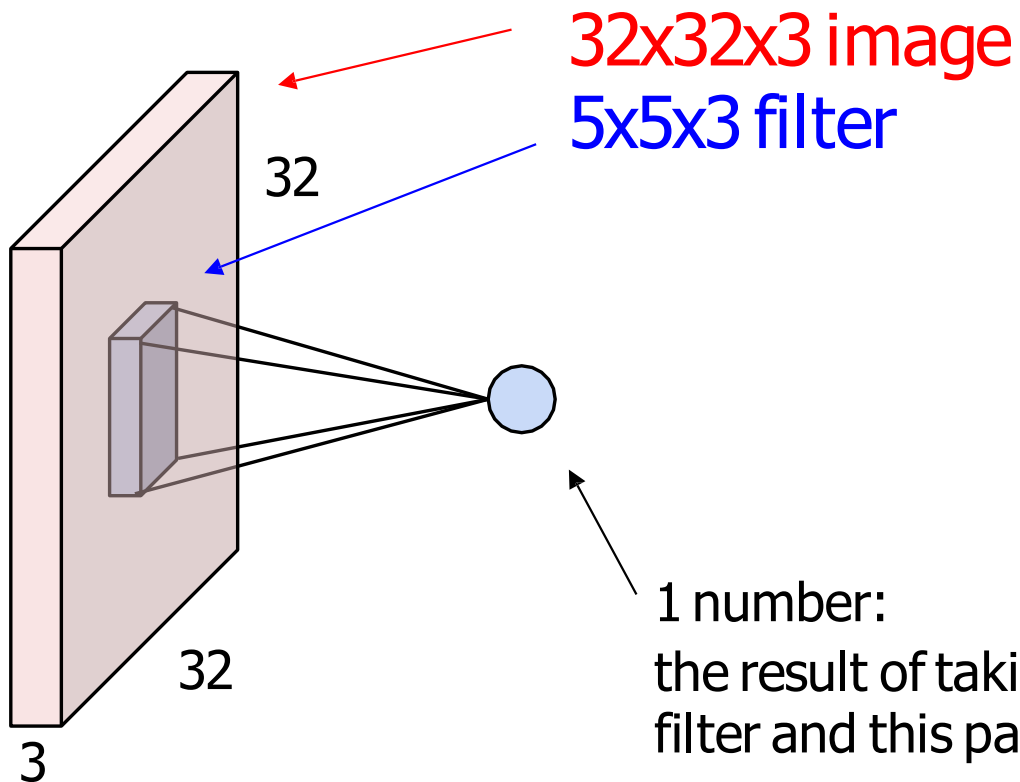
After edge detection filtering



After Relu activation

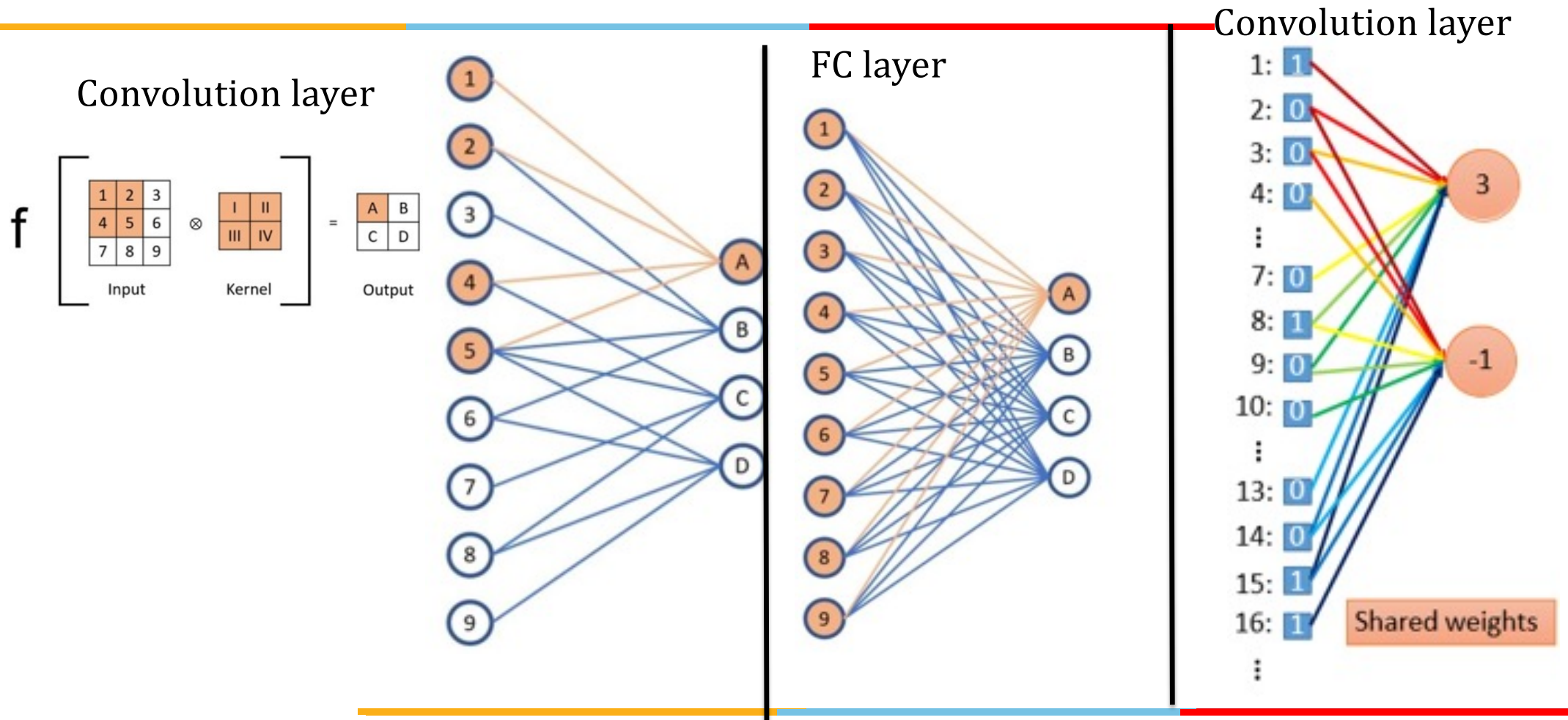


The brain/neuron view of CONV Layer

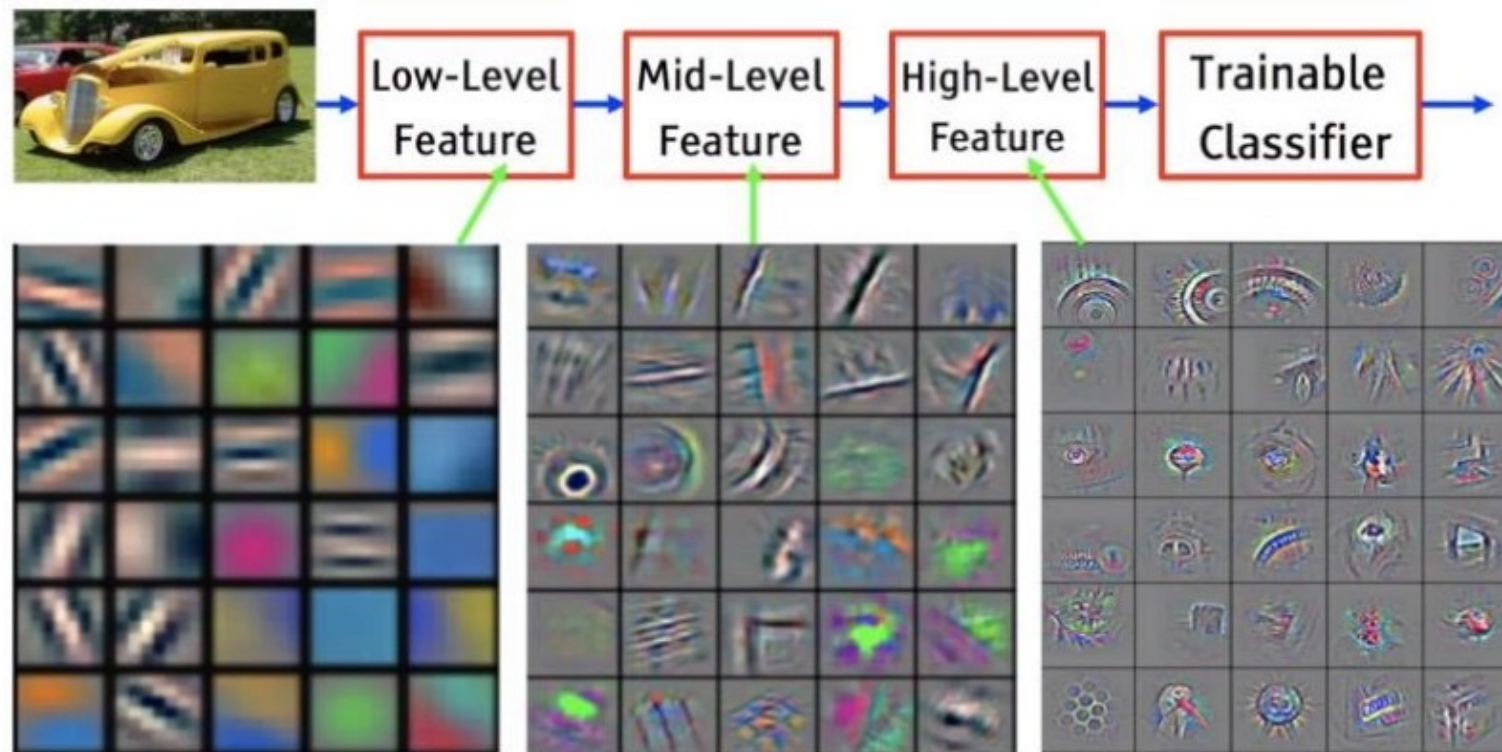


It's just a neuron with local connectivity...

Parameter sharing in Convolution layer



Local to global



CNN – Convolution with Stride



Instead of sliding the kernel over the image by a single pixel, it can be done by any count of pixels. This count is called the **stride**

stride $s = 3$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6×6

Filter1

1	-1	-1
-1	1	-1
-1	-1	1

Output

3	-1
3	-1

Input Size : $n \times n$

Stride : s

Kernel Size : $f \times f$

Output Size : $\left\lceil \frac{n-f}{s} + 1 \right\rceil \times \left\lceil \frac{n-f}{s} + 1 \right\rceil$

CNN – Convolution with Padding



- When filter is convolved with the input image
 - Image shrinks – 6×6 image became a 4×4 image.
 - Pixels in the corners are used only once, when compared the pixels in the middle. The data in the corners are thrown away.
- To resolve, apply padding to the input image before convolution. Padding means append zeros around the boundary of the input.

Two types of padding

- **Valid Padding** : Padding is not applied, $p = 0$
 - Input size $\neq$ output size.
 - Output shrinks when compared to the input.
- **Same Padding** : Padding is applied, $p = \frac{f-1}{2}$
 - Input size = output size.

Input Image with padding 8×8

0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	1	0	0	0	1	0	0
0	0	1	0	0	1	0	0
0	0	0	1	0	1	0	0
0	0	0	0	0	0	0	0

How much to pad?

Kernel 3×3 : $p = 1$

Kernel 5×5 : $p = 2$

Kernel 7×7 : $p = 3$

Dimensions



Input Size : $n \times n$

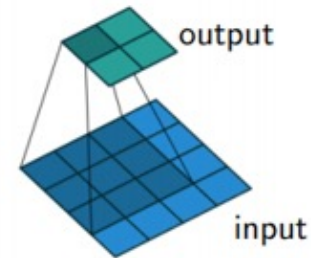
Stride : s

Padding: p

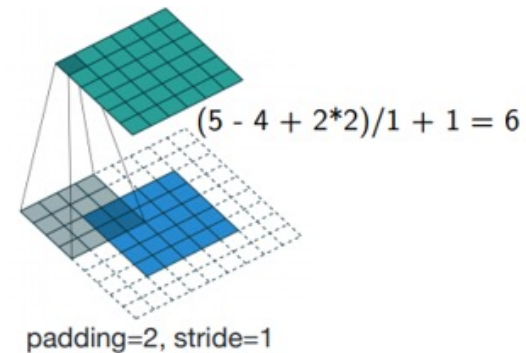
Kernel Size : $f \times f$

Output Size : $\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$

$$(4 - 3 + 2*0)/1 + 1 = 2$$



No padding, stride=1



$$(5 - 4 + 2*2)/1 + 1 = 6$$

Image scale invariance and Pooling



Object can appear anywhere in the image.

Image A

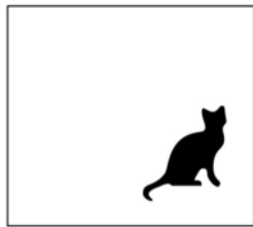


Image B



Model predicts "cat" for both images!

- Helps to achieve image invariance.
- Objects can appear anywhere in the Image and still get detected.
- Used for sub-sampling
- Reduce the size of representation
- Speed up computation
- Two types: Max pooling and Average pooling

Pooling



Input 4×4

$f = 2$

$s = 2$

2	4	7	4
6	6	9	8
3	4	8	3
7	5	3	6



Max pooling

Take the maximum of the sub region that is considered.

Output 2×2

6	9
7	8

Average pooling

Compute the average of the sub region that is considered

Output 2×2

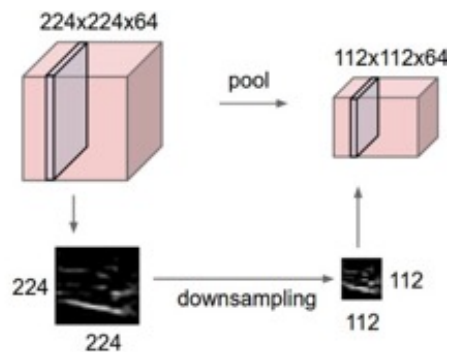
4.5	7
4.75	5

If the input to the pooling layer has multiple channels, then the similar max pooling procedure is applied on all the input slices (channels) and those many slices (channels) will be present in the output as there are in the input.

Convolution + Max Pooling



Volume depth is preserved



Input Image 6×6

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Filter 1

1	-1	-1
-1	1	-1
-1	-1	1

Filter 2

-1	1	-1
-1	1	-1
-1	1	-1

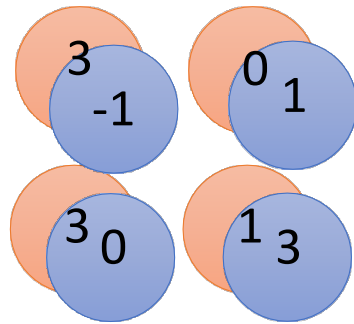
Convolve $s=1$

3	-1	-3	-1
-3	-1	0	-3
-3	-2	0	1
3	-2	-2	-1
-1	-1	-1	-1
-1	-1	-2	-1
-1	-1	-2	1
-1	0	-4	3

Maxpool $f=2, s=2$

3	0
3	1
-1	-1
0	3

Convolution + Max Pooling



A new image

Smaller than the original image

The number of the channel is the number of filters

Convolution

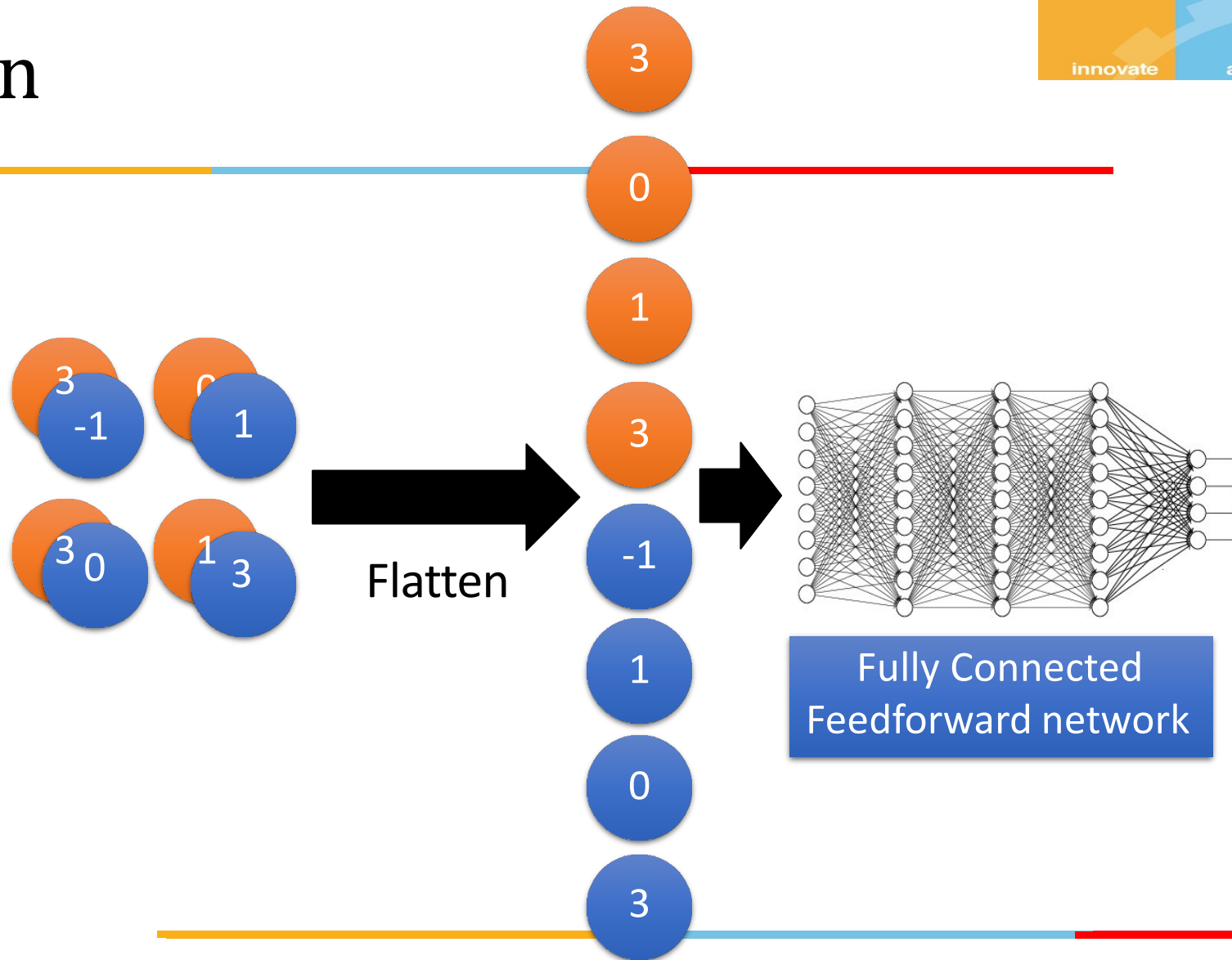
Max Pooling

Convolution

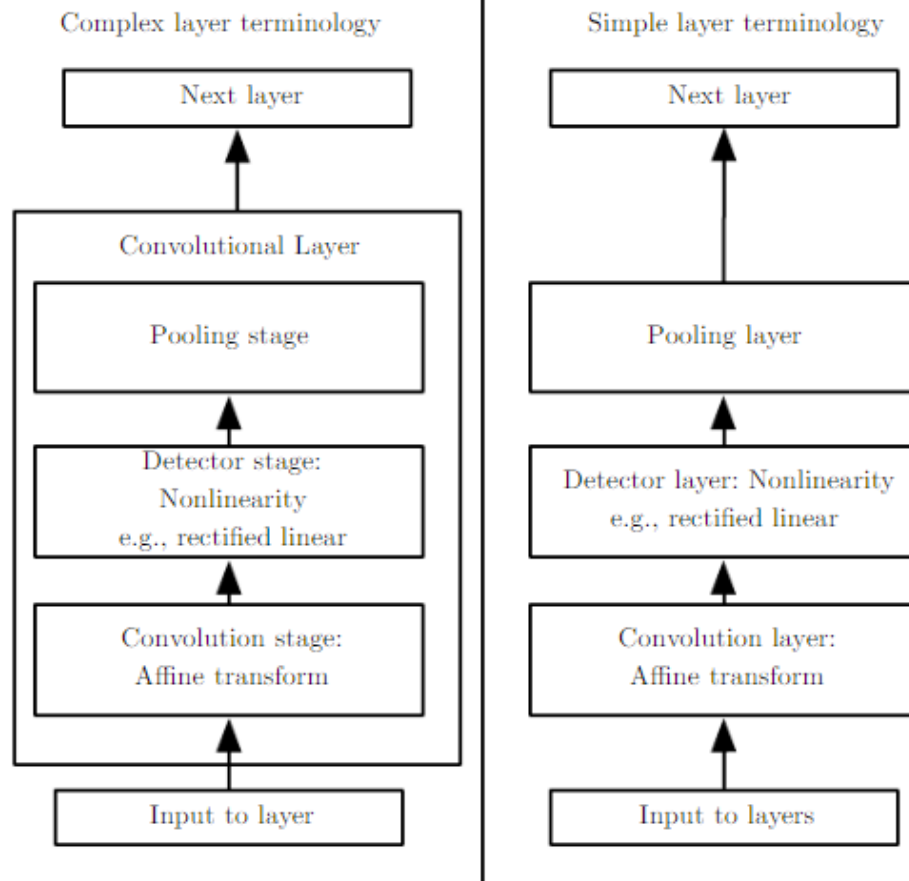
Max Pooling

Can repeat many times

Flatten



CNN layers

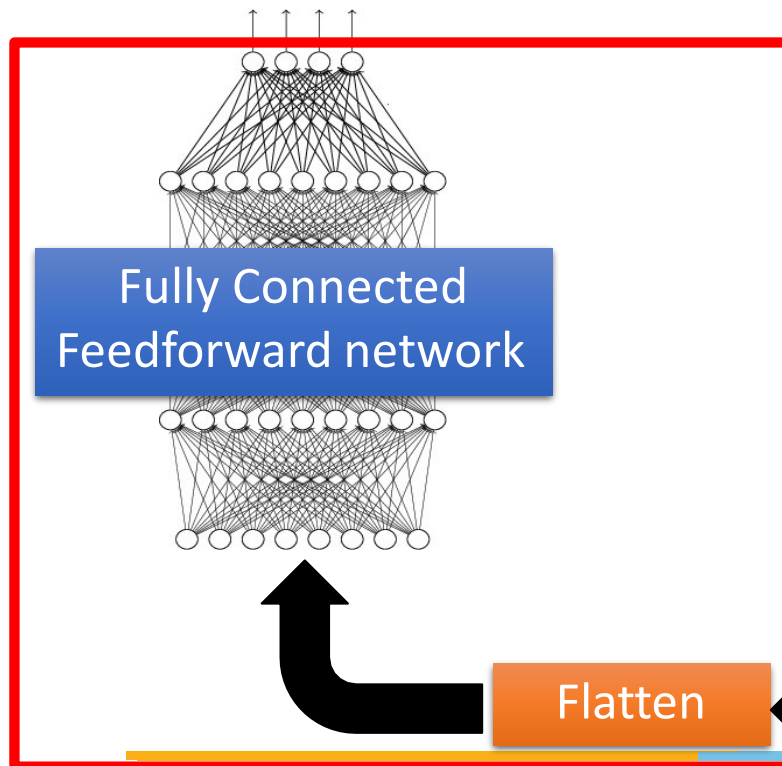


- Every step of processing is regarded as a layer in its own right
- Not every “layer” has parameters

The whole CNN



cat dog



Convolution

Max Pooling

A new image

Convolution

Max Pooling

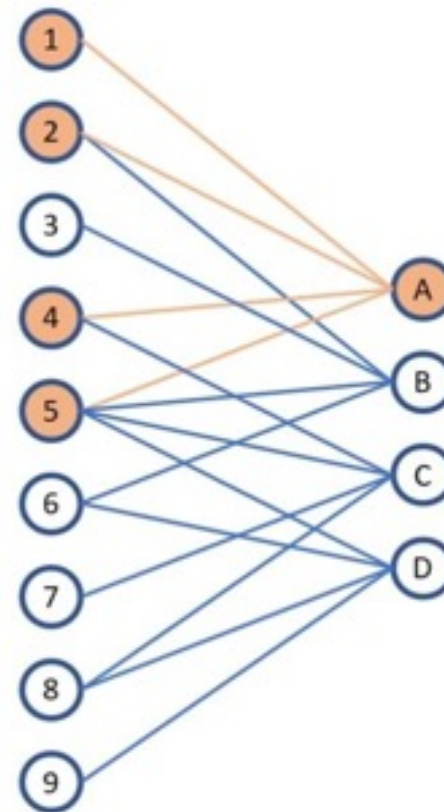
A new image

Flatten

Training



- An input of size 3×3 is convoluted with a filter of size 2×2 with 0 pad and stride of 1 so that it provides an output of size 2×2
- Each neuron on the first layer is receiving each input x_1 to x_9 but weights for many inputs are effectively 0
- Weight matrix (W_1) is a sparse version of the filter.
- Gradients of loss with respect to weights can be calculated as it was done for the fully connected neural networks
- Essentially *CNN is also a MLFF Network*



x_1	x_2	x_3
x_4	x_5	x_6
x_7	x_8	x_9

 *

w_1	w_2
w_3	w_4

Receptive Fields



For convolution with **kernel size K**, each element in the output depends on a $K \times K$ receptive field in the input

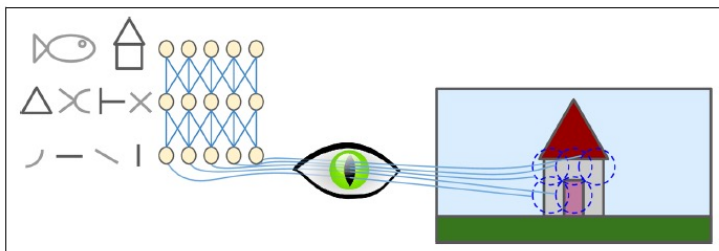
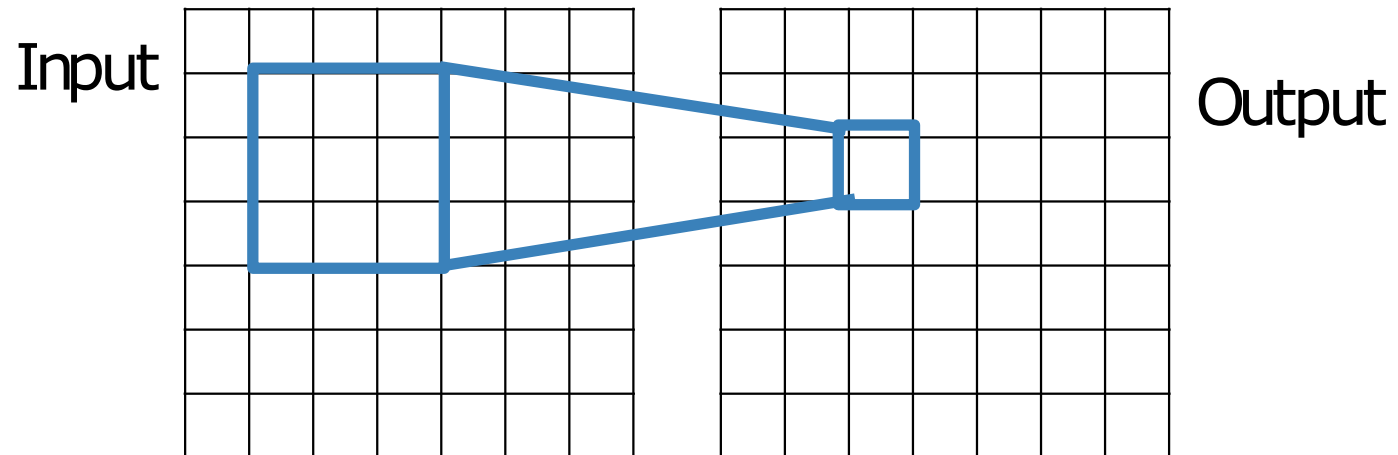
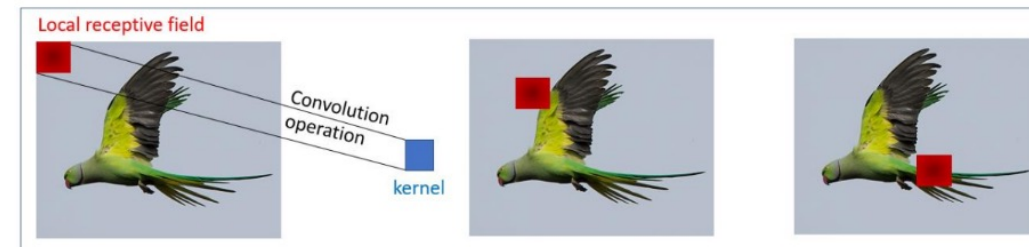


Figure 13-1. Local receptive fields in the visual cortex



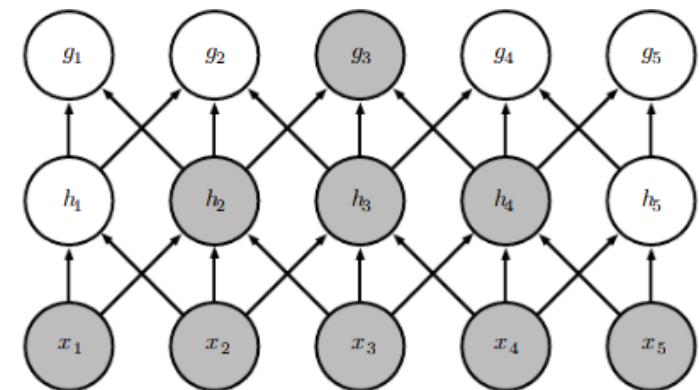
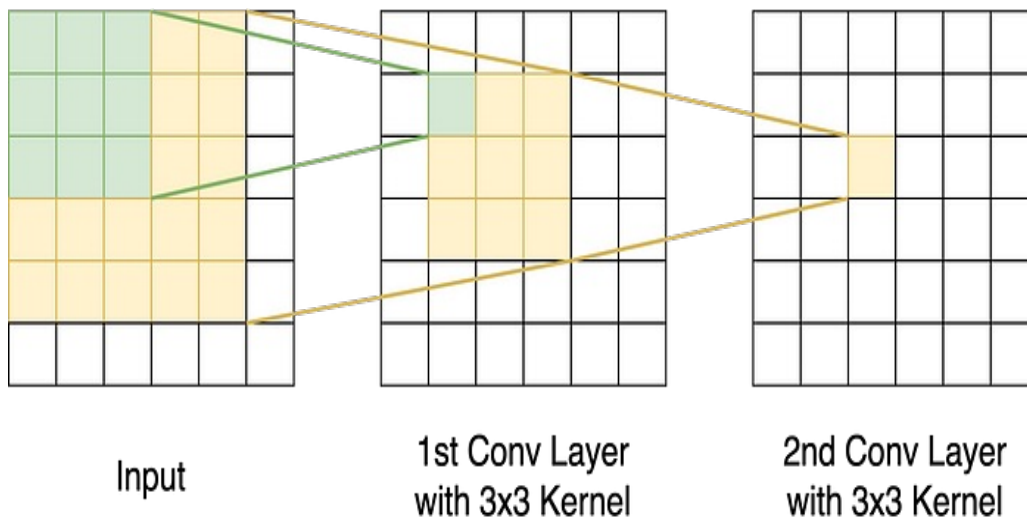


Receptive Fields – deeper layers

The receptive field at layer k is the area denoted $R_k \times R_k$ of the input that each pixel of the k^{th} activation map can 'see'. F_j = filter size of layer j and S_i = stride value of layer i and with the convention $S_0=1$, the receptive field at layer k can be computed with the formula:

$$R_k = 1 + \sum_{j=1}^k (F_j - 1) \prod_{i=0}^{j-1} S_i$$

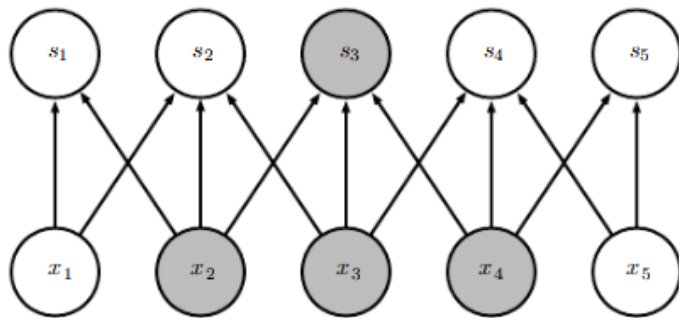
*$F_1=F_2=3$ and $S_1=S_2=1$, $k=2$, which gives $R_2=1+2*1+2*1=5$*



Sparse connectivity in CNN

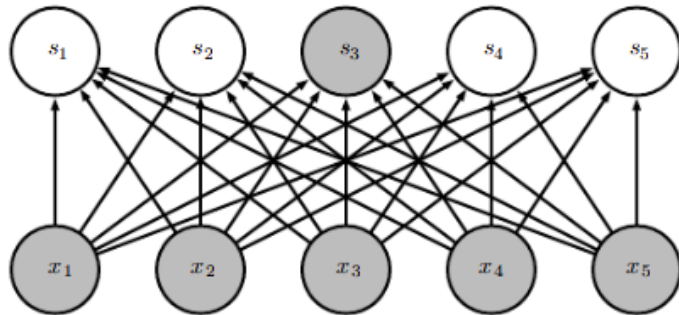


CNN



x_2, x_3, x_4 are receptive fields of s_3

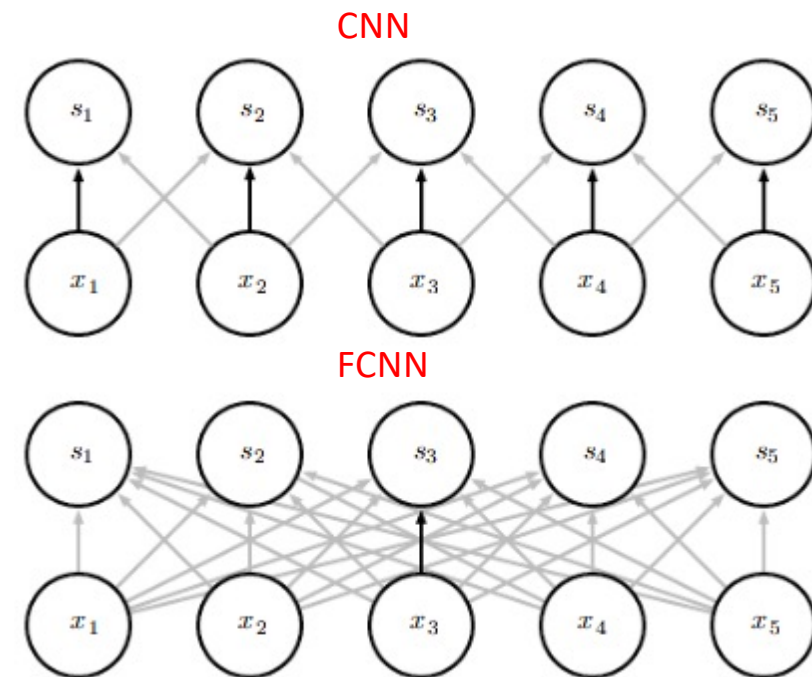
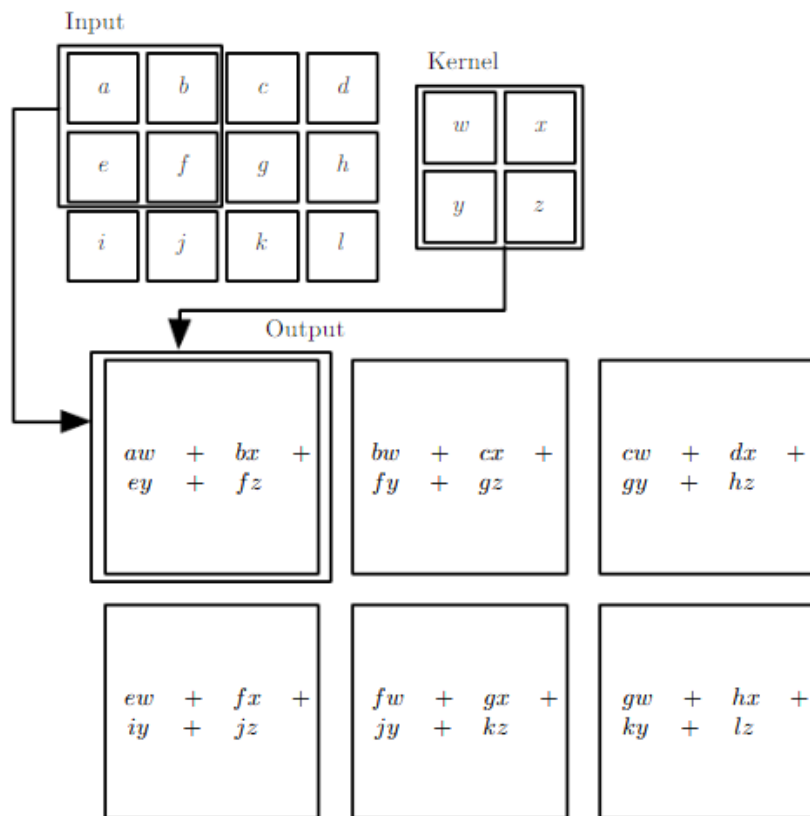
FCNN



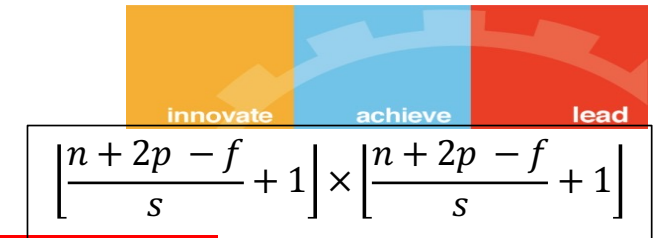
Parameter sharing



The kernel parameters are shared, significant reduction in the memory footprint of the model



Output Size in Convolution Layer



O = Size (width) of output image

I = Size (width) of input image

K = Size (width) of kernels used in the Conv Layer

N = Number of kernels

S = Stride of the convolution operation

P = Padding

The size (O) of the output image is given by $\left\lfloor \frac{I-K+2p}{s} + 1 \right\rfloor$

Example: Example: In AlexNet, the input image is of size 227x227x3. The first convolutional layer has 96 kernels of size 11x11x3 and S =4 , P=0.

$$\left\lfloor \frac{227 - 11 + 2 \times 0}{4} + 1 \right\rfloor = 55$$

Output image is of size **55 x 55 x 96**

Parameter Size in Convolution Layer



K = Size (width) of kernels used in the Conv Layer

N = Number of kernels

C = Number of channels of the input image

W_c = Number of weights of the Conv Layer $\rightarrow W_c = K \times K \times C \times N$

B_c = Number of biases of the Conv Layer $\rightarrow B_c = N$

P_c = Number of parameters of the Conv Layer $\rightarrow P_c = W_c + B_c$

Example: In AlexNet, at the first Conv Layer, the number of channels (C) of the input image is 3, the kernel size (K) is 11, the number of kernels (N) is 96

$$W_c = 11 \times 11 \times 3 \times 96 = 34848$$

$$B_c = 96$$

$$P_c = 34848 + 96 = 34944$$

**The depth of every kernel is always equal to the number of channels in the input image.
So every kernel has $K \times K \times C$ parameters**

Output Size in Pooling Layer



O = Size (width) of output image

I = Size (width) of input image

S = Stride of the convolution operation

P = Padding

The size – (O) of the output image is given by $\left\lfloor \frac{I-K+2p}{s} + 1 \right\rfloor$

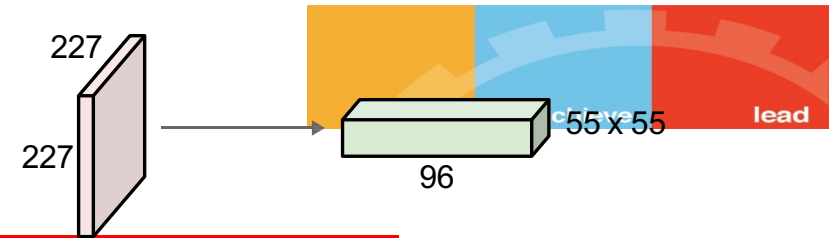
Example: Example: In AlexNet, the MaxPool layer after the bank of convolution filters has a pool size(kernel size) of 3 and stride of 2. We know from the previous section, the image at this stage is of size 55x55x96

$$O = \left\lfloor \frac{55-3+2 \times 0}{2} + 1 \right\rfloor = 27$$

The output size = $O \times O \times C = 27 \times 27 \times 96$

There are no parameters associated with a MaxPool layer. The pool size, stride, and padding are hyperparameters

Example



Consider the three layers of CNN through which an RGB image of size 227x227 is processed. First convolution layer with 96 filters of height and width 11 each, with stride value 4, valid padding and ReLU activation function. Second max pooling layer with 3x3 filter size, with stride value 2, valid padding and no activation function. Dinaly layer is FC for 10 class classification problem. Find the dimension for the output of each layer and total parameters to be learnt. [INPUT - CONV - RELU - POOL - FC]

INPUT	CONV	RELU	POOL	FC
227x227x3	Filters : 96 Filter size:11 x 11 x3 Feature maps : <u>55x55x96</u>	Feature maps : <u>55x55x96</u>	pool size: 3 x 3 Feature maps : <u>27x27x96</u>	Input: 27x27x96 Neurons: 27x27x96 Output: 1x1x10

Parameters

	Filter Count	Filter Size	Weights	Biases	TOTAL
Convolution	96		11 x 11 x 3 x 96 = 34,848	96	34,944
Max Pooling	NA	3 x 3 x 96	0	0	0

Cross-correlation Vs Convolution



- Cross-correlation is sliding dot product over the images
- Convolution flips the kernel and then performs the dot product
- For symmetrical kernels, both operations give the same output
- Initial inception of CNNs is considered to be originated from the paper **Neocognitron**, which uses convolutional filtering
- Most ML/DL packages are known to use correlation filtering instead of convolutional filtering, but still use the name Convolutional Neural Networks
- In deep learning, kernel is learnt during training, thus does not matter whether kernel is flipped or not

Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position

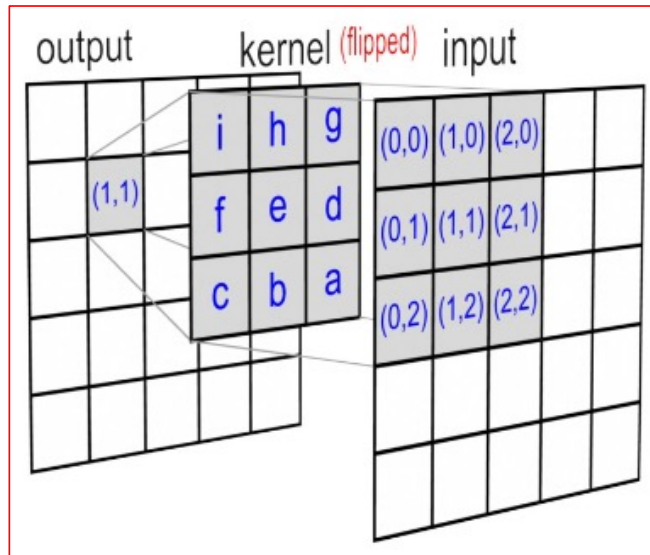
Cross-correlation

$$S_{ij} = \sum_{a=1}^m \sum_{b=1}^n I_{i+a-1, j+b-1} K_{a,b}$$

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n)$$

Convolution is commutative, meaning we can equivalently write

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n).$$



Convolution

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Many machine learning libraries implement cross-correlation but call it convolution

Convolution operation as matrix operation



convolution could be represented as a sparse matrix w where the non-zero elements are the elements w_{ij} of the kernel (with i and j being the row and column of the kernel respectively)

$$\mathbf{W} = \begin{bmatrix} w_{0,0} & w_{0,1} & w_{0,2} \\ w_{1,0} & w_{1,1} & w_{1,2} \\ w_{2,0} & w_{2,1} & w_{2,2} \end{bmatrix} \in \mathbb{R}^{3 \times 3} \quad \mathbf{I} = \begin{bmatrix} i_{0,0} & i_{0,1} & i_{0,2} & i_{0,3} \\ i_{1,0} & i_{1,1} & i_{1,2} & i_{1,3} \\ i_{2,0} & i_{2,1} & i_{2,2} & i_{2,3} \\ i_{3,0} & i_{3,1} & i_{3,2} & i_{3,3} \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

$$\mathbf{W} * \mathbf{I} = \mathbf{O} \in \mathbb{R}^{2 \times 2}$$

$$\mathbf{I}' = [i_{0,0} \ i_{0,1} \ i_{0,2} \ i_{0,3} \ i_{1,0} \ i_{1,1} \ i_{1,2} \ i_{1,3} \ i_{2,0} \ i_{2,1} \ i_{2,2} \ i_{2,3} \ i_{3,0} \ i_{3,1} \ i_{3,2} \ i_{3,3}]^T$$

$$\mathbf{C} \circ \mathbf{I}' = \mathbf{O} \in \mathbb{R}^4$$

$$\mathbf{C} = \begin{pmatrix} w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 & 0 \\ 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} & 0 \\ 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & w_{0,2} & 0 & w_{1,0} & w_{1,1} & w_{1,2} & 0 & w_{2,0} & w_{2,1} & w_{2,2} \end{pmatrix}$$

Convolution operation as matrix operation example



Stride=1
P=0

Kernel

0	1	2
2	2	0
0	1	2

Input

3	3	2	1
0	0	1	3
3	1	2	2
2	0	0	2

stride 1

0	1	2	0
2	2	0	0
0	1	2	0
0	0	0	0

stride 2

0	0	1	2
0	2	2	0
0	0	1	2
0	0	0	0

stride 3

0	0	0	0
0	1	2	0
2	2	0	0
0	1	2	0

stride 4

0	0	0	0
0	0	1	2
0	2	2	0
0	0	1	2

0	1	2	0	2	2	0	0	0	1	2	0	0	0	0	0
0	0	1	2	0	2	2	0	0	0	1	2	0	0	0	0
0	0	0	0	0	0	1	2	0	2	2	0	0	0	1	2
0	0	0	0	0	0	0	1	2	0	2	2	0	0	0	1

.

3
3
2
1
0
0
1
3
3
1
2
2
2
0
0
2

=

12
12
10
17



Output

12	12
10	17

Transposed or fractionally strided convolutions



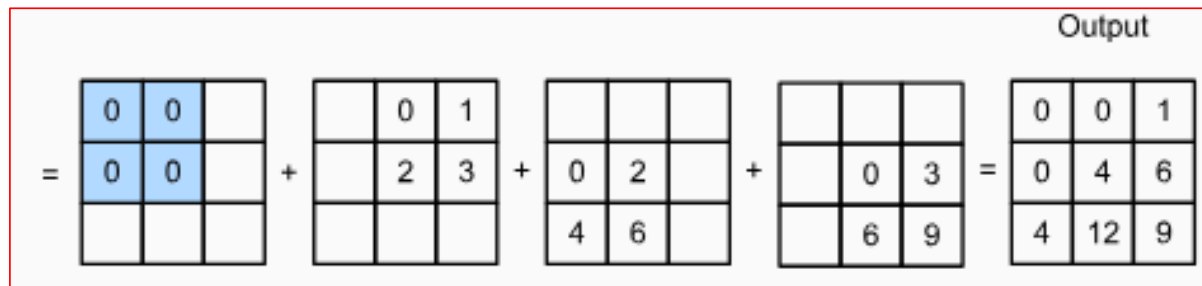
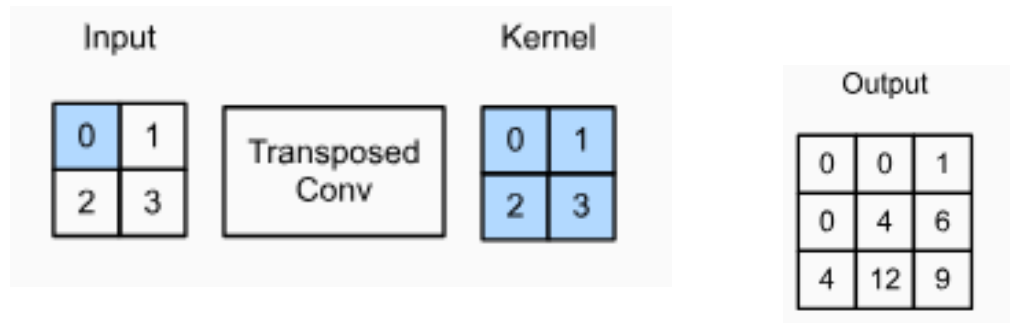
- A **transposed convolutional** is used for up sampling that generates the output feature map greater than the input feature map
- Instead of sliding kernel over the input and performing element-wise multiplication and summation, **a transposed convolutional slides input over kernel and performs element-wise multiplication and summation**
- Applications: image generation, image super-resolution, and image segmentation, converting a low-resolution image to a high-resolution, generating an image from a set of noise vectors

f = convolutional layer , $Y=f(X)$

g = transposed convolutional layer, $g(Y)$ will have the same shape as X

g and f have same hyperparameters

Example – Transposed convolution



input image $\rightarrow n_h \times n_w$

Kernel $\rightarrow f_h \times f_w$

$P_o \rightarrow$ output padding

$$o = (n - 1)s + f - 2p + p_o$$

Transposed convolution as direct convolution



Regular Convolution:

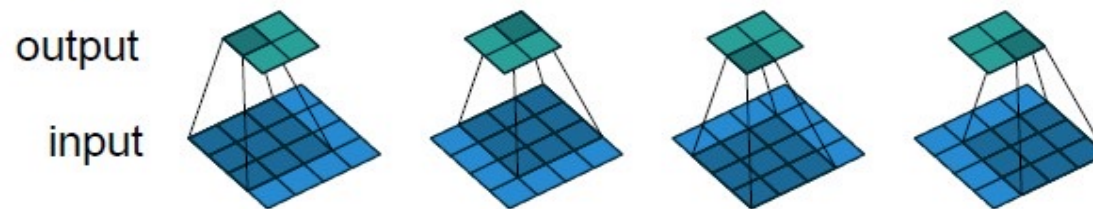
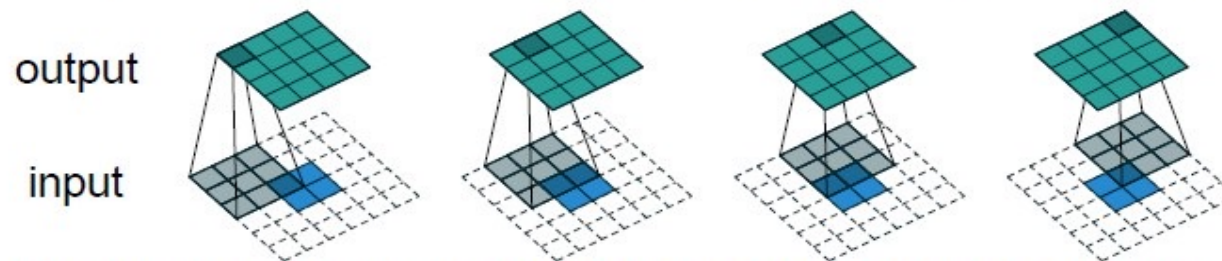


Figure 2.1: (No padding, unit strides) Convolving a 3×3 kernel over a 4×4 input using unit strides (i.e., $i = 4$, $k = 3$, $s = 1$ and $p = 0$).

Transposed Convolution (emulated with direct convolution):



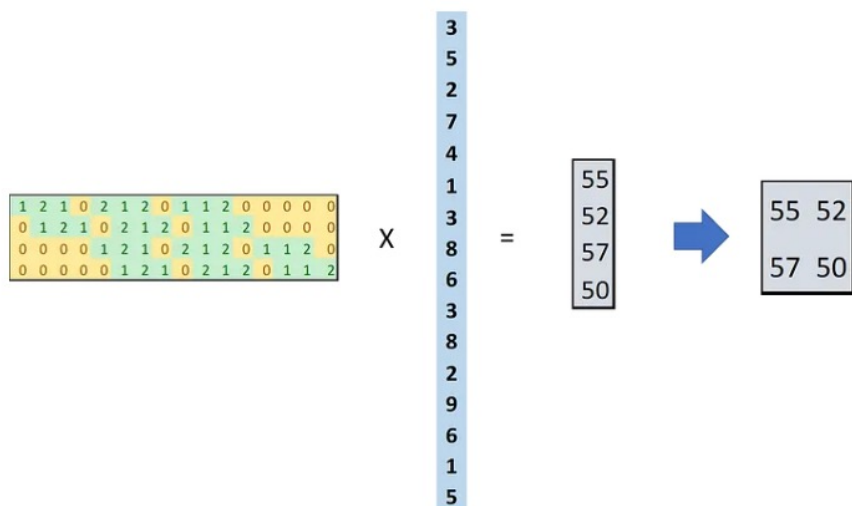
Dumoulin, Vincent, and Francesco Visin. "[A guide to convolution arithmetic for deep learning](#)." *arXiv preprint arXiv:1603.07285* (2016).

Example



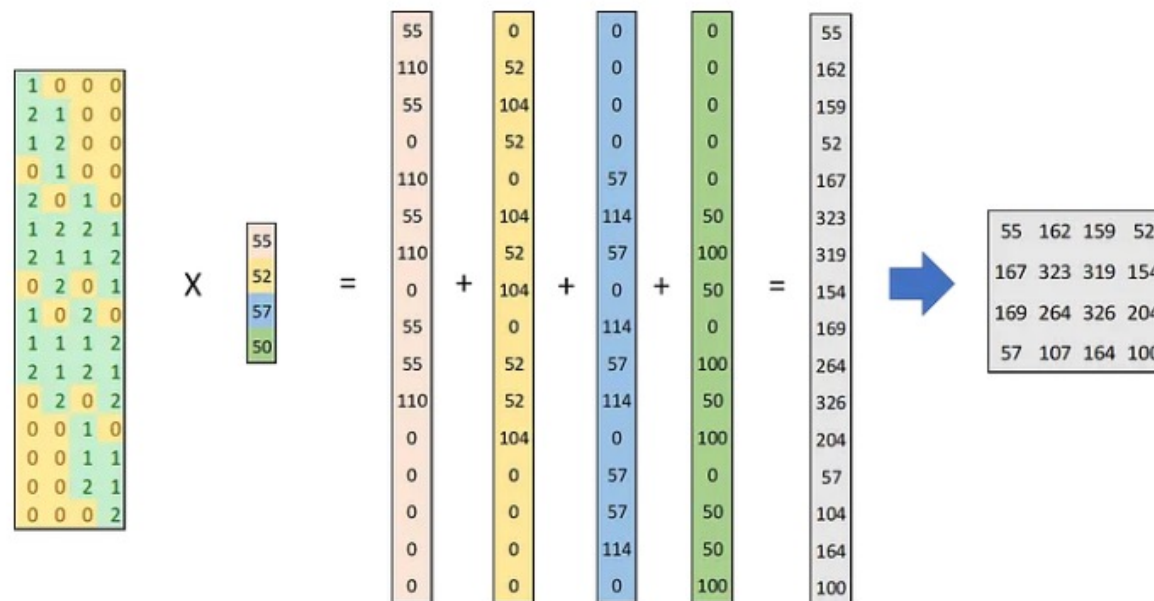
Convolution as matrix operation

$$C \circ I = O$$



Transpose Convolution as matrix operation

$$C^T \circ O = I$$



Dilated convolution



$$O = \left\lfloor \frac{I - K + 2p}{s} + 1 \right\rfloor$$

- **Used to cheaply increase receptive field of output units without increasing kernel size**
- “Inflate” the kernel by inserting spaces between the kernel elements.
- The dilation “rate” is controlled by an additional hyperparameter d .
- Usually $d-1$ spaces inserted between kernel elements ($d = 1$ is a regular convolution)

$$\hat{k} = k + (k - 1)(d - 1)$$

$$o = \left\lfloor \frac{i + 2p - k - (k - 1)(d - 1)}{s} \right\rfloor + 1$$

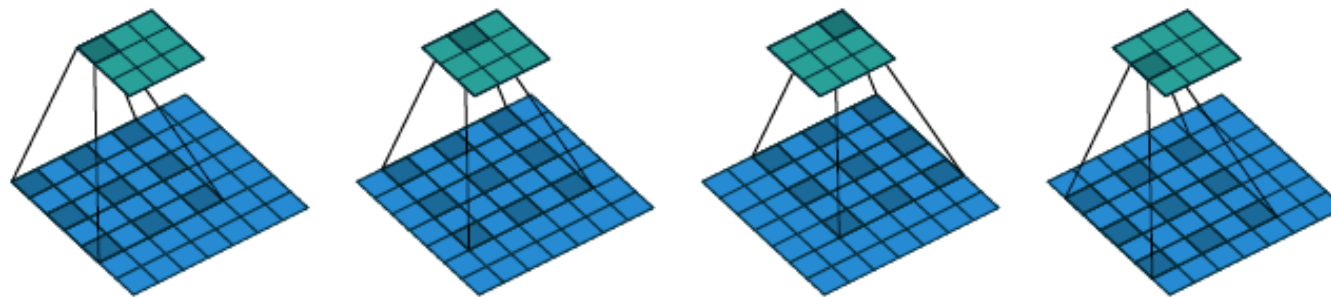


Figure 5.1: (Dilated convolution) Convoluting a 3×3 kernel over a 7×7 input with a dilation factor of 2 (i.e., $i = 7$, $k = 3$, $d = 2$, $s = 1$ and $p = 0$).

Loss/Optimization/Classification



- Binary Cross Entropy, Multi class Cross Entropy, Class-weighted cross-entropy losses (as a way to counter imbalance) L1 (MAE), L2 (MSE) losses
- Dropout
- Batch normalization
- ADAM, RMSProp etc. optimizers
- Sigmoid or softmax classification in output layer



Why CNN for Image

Why CNN for Image



Some patterns are much smaller than the whole image

A neuron does not have to see the whole image to discover the pattern.

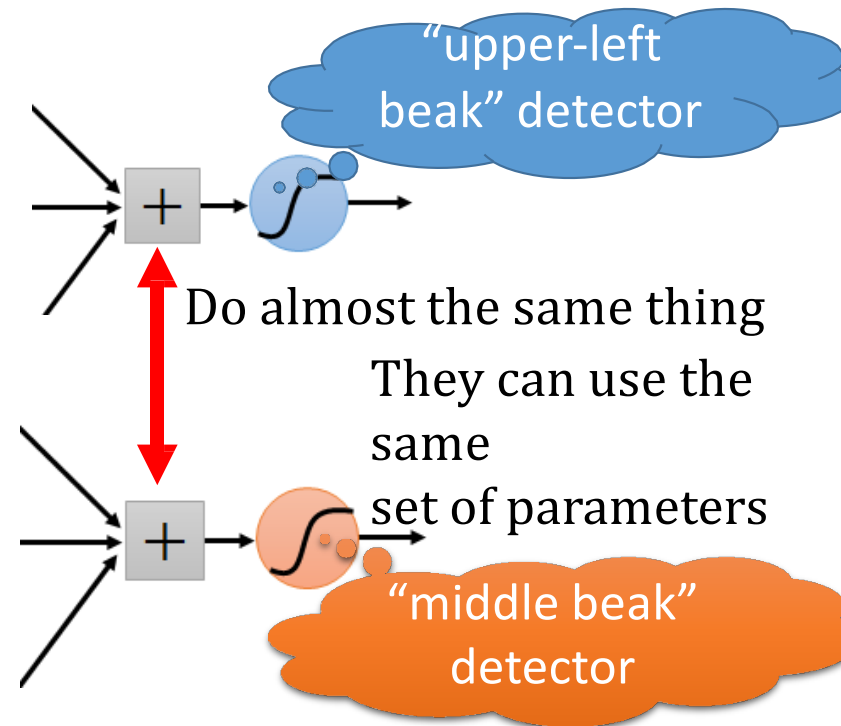
Connecting to small region with less parameters



Why CNN for Image



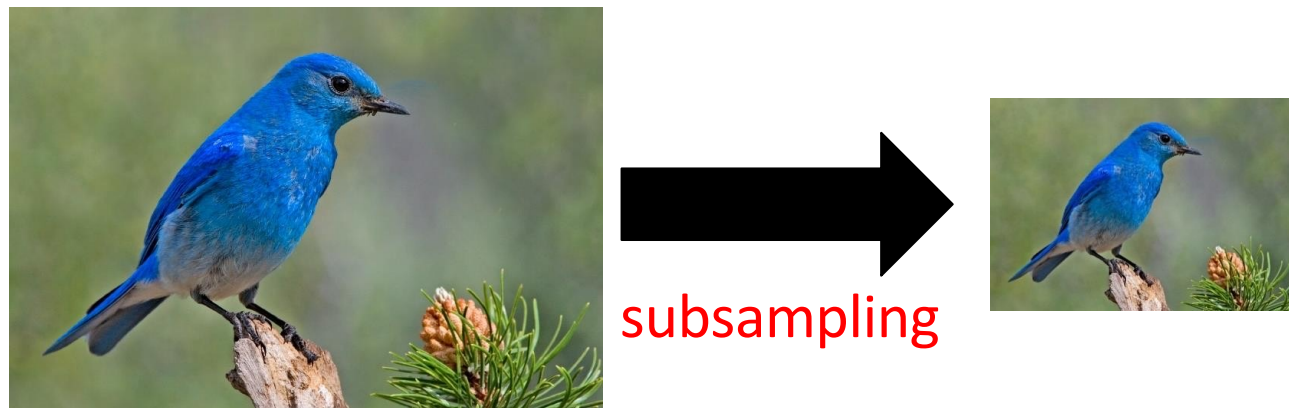
- The same patterns appear in different regions.



Why CNN for Image



Subsampling the pixels will not change the object



We can subsample the pixels to make image smaller

➡ Less parameters for the network to process the image

Why CNN for Image



Property 1

- Some patterns are much smaller than the whole image

Property 2

- The same patterns appear in different regions.

Property 3

- Subsampling the pixels will not change the object

Convolution

Max Pooling

Convolution

Max Pooling

Can repeat many times

Flatten



Practice Questions



- An input of size $m \times n = 5 \times 5$ is convoluted using a filter of $f \times f = 3 \times 3$ with the same padding and stride of 1. What is p ?
Ans : $p=1$
Same padding means, after the convolution the output should be of the same size as it was in the input.
- An input of size $m \times n = 5 \times 6$ is convoluted using a filter of $f \times f = 3 \times 3$ with the same padding and stride of 1. Show the input after padding
Ans : $p=1$
Same padding means, after the convolution the output should be of the same size as it was in the input
- An input of size $m \times n = 5 \times 6$ is convoluted using a filter of 3×3 with the valid padding and stride of 2. What would be the size of output ($a \times b$) after convolution?
Ans: Output size $\rightarrow 2 \times 2$
Valid padding means, $p = 0$

Practice Questions



- Convolution is applied on the input of size $5 \times 5 \times 16$. The filter size is 5×5 , stride is 1 and valid padding is applied with 80 such filters. What would be the shape of the output?

Ans: Valid padding means, $p = 0$.

Output size = $1 \times 1 \times 80$

- Max pooling is applied on an input of size $28 \times 28 \times 8$ with stride value 2, filter size 2×2 and valid padding. What would be the shape of the output?

Ans: Valid padding means, $p = 0$.

Output size = $14 \times 14 \times 8$

References



- Deep Learning by Ian Goodfellow, YoshuaBengio, Aaron Courville
<https://www.deeplearningbook.org/>
- Dive into Deep Learning book
- Deep Learning with Python by Francois Chollet.
<https://livebook.manning.com/book/deep-learning-with-python/>